LeetSolve — Guía de Resolución de los Ejercicios Diarios

Basada en la estructura de 7 escenas y secciones de `GUIA.md` e `INSTRUCCIONES_IA.md`

Periodo: Últimas 2 semanas (9 al 23 de octubre de 2026)

---

1. Contexto: Ejercicios ya realizados en el repositorio

Antes de abordar los ejercicios diarios de las últimas dos semanas, este es el estado actual de los videos en `LeetSolve`:

- **Rama `main` (18 videos):**
  - `arrays-hashing/0001-two-sum` (#1 Two Sum — Plantilla aprobada)
  - `linked-list/0002-add-two-numbers` (#2 Add Two Numbers) y `0019-remove-nth-node` (#19 Remove Nth Node)
  - `sliding-window/0003-longest-substring` (#3 Longest Substring Without Repeating Characters)
  - `binary-search/0004-median-two-arrays` (#4 Median of Two Sorted Arrays)
  - `strings/0005-longest-palindrome` (#5), `0006-zigzag-conversion` (#6) y `0008-string-to-integer` (#8)
  - `math/0007-reverse-integer` (#7), `0009-palindrome-number` (#9) y `0060-permutation-sequence` (#60)
  - `two-pointers/0016-three-sum-closest` (#16) y `0018-four-sum` (#18)
  - `backtracking/0017-letter-combinations` (#17 Letter Combinations of a Phone Number)
  - `trees/0102-level-order` (#102), `0124-max-path-sum` (#124) y `0297-serialize-tree` (#297)
  - `bit-manipulation/3644-max-k-sort-permutation` (#**3644 Maximum K to Sort a Permutation — Diario del 23 de oct**)
- **Rama `laptop-videos` (8 videos adicionales):**
  - `#10` Regular Expression Matching, `#11` Container With Most Water, `#12` Integer to Roman, `#13` Roman to Integer, `#14` Longest Common Prefix, `#15` 3Sum, `#1807` Evaluate the Bracket Pairs of a String.
  - `strings/3863-min-ops-sort-string` (#**3863 Minimum Operations to Sort a String — Diario del 21 de oct**).

2. Mapa de los Ejercicios Diarios (9 al 23 de octubre de 2026)

Fecha	# Problema	Dif.	Sección	Idea Clave (El Truco)	Tiempo / Esp.
2026-10-09	#1541 Balance Paren.	Med	`strings`	Conteo voraz de `'('` abiertos y pares `'))'`	$O(n)$ / $O(1)$
2026-10-10	#2333 Min Squared Diff	Med	`math`	Cubetas de frecuencias + bajar el tope	$O(n+M)$ / $O(M)$
2026-10-11	#2778 Special Squares	Easy	`arrays-hashing`	Filtrar divisores $n \bmod i = 0$ en 1 pasada	$O(n)$ / $O(1)$
2026-10-12	#2897 Squares AND/OR	Hard	`bit-manipulation`	Gravedad de bits por columna (30 bits)	$O(30n)$ / $O(1)$
2026-10-13	#3727 Alt. Squares	Med	`math`	Ordenar x^2: mayores suman, menores restan	$O(n \log n)$ / $O(n)$
2026-10-14	#3723 Digits Squares	Med	`math`	Convexidad: llenar con `'9'` de izq. a der.	$O(\text{num})$ / $O(\text{num})$
2026-10-15	#2913 Distinct Sub. I	Easy	`arrays-hashing`	Conjunto incremental fijando el inicio i	$O(n^2)$ / $O(n)$
2026-10-16	#2916 Distinct Sub. II	Hard	`trees`	Segment Tree Lazy con $\sum d$ y $\sum d^2$	$O(n \log n)$ / $O(n)$
2026-10-17	#3551 Swaps Digit Sum	Med	`arrays-hashing`	Descomposición en ciclos: $\sum(\lvert C\rvert - 1)$	$O(n \log n)$ / $O(n)$
2026-10-18	#1569 Ways Same BST	Hard	`trees`	Divide y vencerás + $\binom{L+R}{L}$	$O(n^2)$ / $O(n)$
2026-10-19	#2089 Target Indices	Easy	`binary-search`	Contar menores e iguales sin ordenar	$O(n)$ / $O(1)$
2026-10-20	#3886 Sortable Ints	Hard	`arrays-hashing`	Divisores de $n + \leq 1$ caída circular	$O(n \log n)$ / $O(n)$
2026-10-21	#3863 Sort String*	Med	`strings`	Análisis en $O(n)$: resp. $\in \{0, 1, 2, 3, -1\}$	$O(n)$ / $O(1)$
2026-10-22	#3942 Sort Perm.	Med	`arrays-hashing`	Grupo diedral: rotación directa o invertida	$O(n)$ / $O(1)$
2026-10-23	#3644 Max K Perm.*	Med	`bit-manipulation`	AND bit a bit de bloques fuera de lugar	$O(n)$ / $O(1)$

*Nota: El #**3863** ya está hecho en la rama `laptop-videos` y el #**3644** ya está en `main`. Se incluyen aquí con el mismo diseño del repositorio para tener las 2 semanas completas.

1 2026-10-09 | #1541 Minimum Insertions to Balance a Parentheses String

videos/strings/1541-min-insertions-balance-parentheses [Medium] · **Tema:** `strings` (Stack / Greedy)

Scene1_Hook — Gancho y metáfora en Minecraft

Cada bloque de apertura `'('` es un cofre que exige exactamente **dos llaves consecutivas** `'))'` a su derecha para cerrarse. Si un cierre viene suelto o sin cofre previo, insertamos la pieza faltante al vuelo.

Scene2_Problem — El problema y ejemplo pequeño (6 bloques)

Dada una cadena s con `'('` y `')'`, una cadena está balanceada si cada `'('` se empareja con dos `'))'` consecutivos posteriores. Podemos insertar `'('` o `')'` en cualquier posición. Buscamos el mínimo de inserciones.

- **Ejemplo para fila de 6 bloques:** `s = "))())("` (Menas: `redstone` para `')'` y `diamond` para `'('`).
- **Respuesta esperada:** `3` (insertar `'('` al inicio para el primer `'))'` e insertar `'))'` al final para el último `'('`).

Scene3_Brute — La idea directa (Fuerza bruta) y por qué falla

Intentar insertar caracteres en todas las posiciones posibles mediante búsqueda en anchura (BFS) o programación dinámica de intervalos tipo CYK ($O(n^3)$). Con $n = 10^5$, probar combinaciones o una tabla cuadrática/cúbica rompe el pico de inmediato.

Scene4_Idea — El truco (Un solo recorrido con contador de abiertos)

No necesitamos explorar ramas: cuando vemos un `')'`, sabemos de inmediato si tiene a su gemelo `')'` justo al lado:

1. Si llega un `'('`, sumamos 1 al contador de abiertos `open_left`.
2. Si llega un `')'`, miramos si el siguiente carácter también es `')'`. Si sí, avanzamos 2 casillas (ya forman el par `'))'`); si no, insertamos un `')'` (`ans += 1`) y avanzamos 1 casilla.
3. Ya con el par `'))'` completo, si `open_left > 0` gastamos un `'('` abierto (`open_left -= 1`); si `open_left == 0`, insertamos un `'('` antes (`ans += 1`).
4. Al terminar la fila, cada `'('` que quedó sin cerrar en `open_left` necesita dos cierres: sumamos `2 * open_left`.

Scene5_Walk — Recorrido paso a paso sobre `s = "))())("`

- `i = 0..1` (`"))"`): Hay dos `')'` seguidos, pero `open_left = 0`. Insertamos un `'('` → `ans = 1`, `open_left = 0`.
- `i = 2` (`"("`): Guardamos un abierto en el cofre → `open_left = 1`.
- `i = 3..4` (`"))"`): Par completo; consume el abierto del cofre → `open_left = 0`, `ans = 1`.
- `i = 5` (`"("`): Nuevo abierto → `open_left = 1`.
- **Fin de la fila:** Queda `open_left = 1`, que exige dos cierres `'))'` → `ans = 1 + 2(1) = 3`. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n)$ (una sola pasada por la cadena). **Espacio:** $O(1)$ (dos contadores enteros).

Scene7_Outro — Código en Python y lección

```
class Solution:
    def minInsertions(self, s: str) -> int:
        ans = open_left = i = 0
        n = len(s)
        while i < n:
            if s[i] == "(":
                open_left += 1
                i += 1
```

```
            else:
                if i + 1 < n and s[i + 1] == ")":
                    i += 2
                else:
                    ans += 1
                    i += 1
                if open_left > 0:
                    open_left -= 1
                else:
                    ans += 1
        return ans + 2 * open_left
```

Lección: *"Empareja cada cierre doble al instante y cobra los abiertos pendientes al final."*

2 2026-10-10 | #2333 Minimum Sum of Squared Difference

videos/math/2333-min-sum-squared-diff [Medium] · **Tema:** `math` / `arrays-hashing` (Greedy / Buckets)

Scene1_Hook — Gancho y metáfora en Minecraft

Tenemos torres de bloques cuya altura es la diferencia $d_i = |\text{nums1}[i] - \text{nums2}[i]|$. Cada golpe de pico rebaja una torre en 1 bloque, y el costo de cada torre es el **cuadrado** de su altura.

Scene2_Problem — El problema y ejemplo pequeño (5 bloques)

Dados `nums1` y `nums2` de longitud n, podemos modificar `nums1` hasta k_1 veces en ± 1 y `nums2` hasta k_2 veces en ± 1. Queremos minimizar $\sum_{i=0}^{n-1} (\text{nums1}[i] - \text{nums2}[i])^2$.

- **Ejemplo de 5 bloques:** `nums1 = [1, 4, 10, 12, 2]`, `nums2 = [5, 8, 6, 9, 2]`, $k_1 = 1, k_2 = 2$.
- **Diferencias iniciales:** $d = [4, 4, 4, 3, 0]$ con presupuesto total $k = k_1 + k_2 = 3$.
- **Respuesta esperada:** `36` (las tres diferencias de 4 bajan a 3, dejando $[3, 3, 3, 3, 0] \to 4 \times 3^2 + 0^2 = 36$).

Scene3_Brute — La idea directa (Max-Heap paso a paso) y por qué falla

Meter las diferencias a un Max-Heap y, $k_1 + k_2$ veces, sacar la mayor diferencia d, restarle 1 y volverla a meter. Como $k_1 + k_2$ puede llegar a 2×10^9, hacer 2×10^9 extracciones tarda $O(k \log n)$ y da *Time Limit Exceeded*.

Scene4_Idea — El truco (Aplanar las diferencias más altas por cubetas)

1. Subir `nums1[i]` en 1 o bajar `nums2[i]` en 1 tiene exactamente el mismo efecto: reduce $d_i = |\text{nums1}[i] - \text{nums2}[i]|$ en 1. Por tanto, solo importa el presupuesto conjunto $k = k_1 + k_2$.
2. Bajar una diferencia de d a $d - 1$ ahorra $d^2 - (d-1)^2 = 2d - 1$. Siempre conviene rebajar primero la diferencia **más grande**.
3. Como $d_i \le 10^5$, contamos en un arreglo `cnt[d]` cuántas diferencias valen d y recorremos d desde $d_{\max}$ hasta 1, bajando en un solo paso `take = min(cnt[d], k)` elementos del escalón d al escalón $d - 1$.

Scene5_Walk — Recorrido paso a paso sobre $d = [4, 4, 4, 3, 0]$, $k = 3$

- Conteo en el cofre: `cnt[4] = 3`, `cnt[3] = 1`, `cnt[0] = 1`.
- Escalón $d = 4$: hay 3 bloques y tenemos $k = 3$ golpes. `take = min(3, 3) = 3`. Los 3 bajan al escalón 3: ahora `cnt[4] = 0`, `cnt[3] = 1 + 3 = 4`, y $k = 0$.
- Como $k = 0$, paramos y sumamos: $4 \times 3^2 + 1 \times 0^2 = 36$. *Advancement Made!*

Scene6_Complexity — Complejidad

Sea $M = \max(d_i) \le 10^5$. **Tiempo:** $O(n + M)$. **Espacio:** $O(M)$ para las cubetas de frecuencias.

Scene7_Outro — Código en Python y lección

```
class Solution:
    def minSumSquareDiff(self, nums1: list[int], nums2: list[int], k1: int, k2: int) -> int:
        diffs = [abs(a - b) for a, b in zip(nums1, nums2)]
        mx = max(diffs)
        if mx == 0:
            return 0
        cnt = [0] * (mx + 1)
        for d in diffs:
            cnt[d] += 1
        k = k1 + k2
        for d in range(mx, 0, -1):
```

```
            if k <= 0:
                break
            if cnt[d]:
                take = min(cnt[d], k)
                cnt[d] -= take
                cnt[d - 1] += take
                k -= take
        return sum(d * d * cnt[d] for d in range(1, mx + 1))
```

Lección: *"Cuando los cuadrados castigan a los grandes, rebaja la cima entera nivel por nivel."*

3 2026-10-11 | #2778 Sum of Squares of Special Elements

videos/arrays-hashing/2778-sum-squares-special **[Easy]** · **Tema:** `arrays-hashing` / `math`

Scene1_Hook — Gancho y metáfora en Minecraft

Una fila de n bloques numerados del 1 al n. Solo los bloques cuya posición divide exactamente a n se iluminan y aportan su cuadrado al nivel de experiencia.

Scene2_Problem — El problema y ejemplo pequeño (6 bloques)

Dado un arreglo `nums` de longitud n **indexado desde 1**, un elemento `nums[i]` es especial si i divide a n ($n \bmod i == 0$). Devolver la suma de los cuadrados de todos los elementos especiales.

- **Ejemplo de 6 bloques:** `nums = [2, 7, 1, 19, 18, 3]` ($n = 6$).
- **Divisores de 6:** $i \in \{1, 2, 3, 6\} \implies$ valores $2, 7, 1, 3$.
- **Respuesta esperada:** $2^2 + 7^2 + 1^2 + 3^2 = 4 + 49 + 1 + 9 = 63$.

Scene3_Brute — La trampa común (Error de indexación 0-based vs 1-based)

El error típico es usar el índice 0-based de Python directamente (`n % 0` lanza `ZeroDivisionError`, o evalúa las casillas corridas un lugar).

Scene4_Idea — El truco (`enumerate(nums, 1)` o iterar divisores en $\sqrt{n}$)

Numeramos los bloques empezando en 1 con `enumerate(nums, 1)`. En cada paso preguntamos si `n % i == 0`; si se cumple, sumamos `x * x` al acumulador. (Incluso si n fuera enorme, bastaría iterar i hasta $\lfloor\sqrt{n}\rfloor$ y sumar tanto i como $n//i$).

Scene5_Walk — Recorrido paso a paso sobre `nums = [2, 7, 1, 19, 18, 3]`

- $i = 1$ (valor 2): $6 \bmod 1 = 0$ ✓ → sumamos $2^2 = 4$ (`total = 4`).
- $i = 2$ (valor 7): $6 \bmod 2 = 0$ ✓ → sumamos $7^2 = 49$ (`total = 53`).
- $i = 3$ (valor 1): $6 \bmod 3 = 0$ ✓ → sumamos $1^2 = 1$ (`total = 54`).
- $i = 4$ (valor 19) e $i = 5$ (valor 18): $6 \bmod 4 \neq 0$, $6 \bmod 5 \neq 0$ × → se saltan.
- $i = 6$ (valor 3): $6 \bmod 6 = 0$ ✓ → sumamos $3^2 = 9$ (`total = 63`). *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n)$ (o $O(\sqrt{n})$ iterando divisores). **Espacio:** $O(1)$.

Scene7_Outro — Código en Python y lección

```
class Solution:
    def sumOfSquares(self, nums: list[int]) -> int:
        n = len(nums)
        return sum(x * x for i, x in enumerate(nums, 1) if n % i == 0)
```

Lección: *"Empieza a contar desde uno con `enumerate(nums, 1)` y deja que el módulo filtre los divisores."*

4 2026-10-12 | #2897 Apply Operations on Array to Maximize Sum of Squares

videos/bit-manipulation/2897-max-sum-squares-and-or **[Hard]** · **Tema:** `bit-manipulation` (Greedy)

Scene1_Hook — Gancho y metáfora en Minecraft

Cada número tiene su columna de lámparas de redstone (sus bits). Al aplicar (a AND b, a OR b), las lámparas encendidas nunca se destruyen: simplemente **caen por gravedad** hacia un mismo bloque para volverlo más brillante.

Scene2_Problem — El problema y ejemplo pequeño (5 bloques)

Dado un arreglo `nums` y un entero k, podemos elegir dos índices i, j cualquier número de veces y reemplazar $(\text{nums}[i], \text{nums}[j])$ por $(\text{nums}[i] \text{ AND } \text{nums}[j], \text{nums}[i] \text{ OR } \text{nums}[j])$. Al final elegimos k elementos y maximizamos la suma de sus cuadrados módulo $10^9 + 7$.

- **Ejemplo de 5 bloques:** `nums = [2, 6, 5, 8, 1]`, $k = 2$.
- **En binario (4 bits):** $2 = 0010_2$, $6 = 0110_2$, $5 = 0101_2$, $8 = 1000_2$, $1 = 0001_2$.
- **Respuesta esperada:** `274` (formamos $15 = 1111_2$ y $7 = 0111_2 \implies 15^2 + 7^2 = 225 + 49 = 274$).

Scene3_Brute — La idea directa (Probar pares de operaciones) y por qué falla

Simular pares (i, j) en busca del mejor arreglo final. Con $n = 10^5$, hay $\binom{n}{2} \approx 5 \times 10^9$ pares tan solo en el primer paso.

Scene4_Idea — El truco (Conservación de bits por columna y apilado voraz)

1. Observa qué hace (x AND y, x OR y) en cada bit b: si x e y tienen ambos 1 o ambos 0, nada cambia. Si uno tiene 1 y el otro 0, el 1 se mueve a x OR y y el 0 queda en x AND y.
2. Por tanto, **la cantidad total de unos en cada columna de bit b es invariante**, y podemos transferir bits libremente para concentrarlos en los primeros k números.
3. Como $(A + 2^b)^2 + (B - 2^b)^2 = A^2 + B^2 + 2^{b+1}(A - B + 2^b) > A^2 + B^2$ para $A \geq B$, siempre conviene concentrar todos los bits posibles en el primer número, luego los sobrantes en el segundo, y así hasta k.

Scene5_Walk — Recorrido paso a paso sobre `nums = [2, 6, 5, 8, 1]`, $k = 2$

- Contamos cuántas lámparas hay encendidas por bit $(8, 4, 2, 1)$: `cnt[3] = 1` (del 8), `cnt[2] = 2` (de $6, 5$), `cnt[1] = 2` (de $2, 6$), `cnt[0] = 2` (de $5, 1$).
- **Casilla 1 (idx $= 0$):** Toma un bit de cada columna con `cnt`$[b] > 0 \implies 8 + 4 + 2 + 1 = 15$, aporta $15^2 = 225$.
- **Casilla 2 (idx $= 1$):** Toma un bit de cada columna con `cnt`$[b] > 1 \implies 0 + 4 + 2 + 1 = 7$, aporta $7^2 = 49$.
- **Suma final:** $225 + 49 = 274$. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(30 \cdot n)$ (lineal en n). **Espacio:** $O(1)$ (un arreglo fijo de 30 contadores).

Scene7_Outro — Código en Python y lección

```
class Solution:
    def maxSum(self, nums: list[int], k: int) -> int:
        MOD = 10**9 + 7
        cnt = [0] * 30
        for x in nums:
            for b in range(30):
                if (x >> b) & 1:
                    cnt[b] += 1
```

```
        ans = 0
        for idx in range(k):
            val = 0
            for b in range(30):
                if cnt[b] > idx:
                    val |= 1 << b
            ans = (ans + val * val) % MOD
        return ans
```

Lección: *"AND y OR no crean ni destruyen bits: solo dejan caer todos los unos hacia los primeros k bloques."*

5 2026-10-13 | #3727 Maximum Alternating Sum of Squares

videos/math/3727-max-alternating-sum-squares **[Medium]** · **Tema:** `math` (Greedy / Sorting)

Scene1_Hook — Gancho y metáfora en Minecraft

Una fila de casillas donde las posiciones pares suman esmeraldas ($+x^2$) y las impares quitan redstone ($-x^2$). Para ganar el máximo puntaje, ponemos los cuadrados más grandes donde suman y los más pequeños donde restan.

Scene2_Problem — El problema y ejemplo pequeño (6 bloques)

Dado un arreglo entero `nums` (con positivos y negativos), podemos reordenarlo en cualquier orden para maximizar:

$$\text{score} = \text{arr}[0]^2 - \text{arr}[1]^2 + \text{arr}[2]^2 - \text{arr}[3]^2 + \dots$$

- **Ejemplo de 6 bloques:** `nums = [1, -4, 2, -3, 5, -2]`.
- **Cuadrados:** $[1, 16, 4, 9, 25, 4]$.
- **Respuesta esperada:** `41` (suman $25 + 16 + 9 = 50$, restan $4 + 4 + 1 = 9 \implies 50 - 9 = 41$).

Scene3_Brute — La idea directa (Probar todas las permutaciones) y por qué falla

Probar las $n!$ formas de acomodar los bloques y calcular el puntaje alternante de cada una. Con $n = 6$ ya son 720 pruebas; con $n = 10^5$ es inviable ($O(n! \cdot n)$).

Scene4_Idea — El truco (Ordenar los cuadrados y partir en $\lfloor n/2 \rfloor$)

1. Al elevar al cuadrado, el signo original no importa: $(-4)^2 = 4^2 = 16$.
2. En un arreglo de longitud n, hay exactamente $\lfloor n/2 \rfloor$ posiciones con signo $-$ (los índices impares $1, 3, 5, \dots$) y $\lceil n/2 \rceil$ posiciones con signo $+$ (los índices pares $0, 2, 4, \dots$).
3. Ordenamos los cuadrados de menor a mayor: los primeros $m = \lfloor n/2 \rfloor$ van a las posiciones que restan y los restantes $n - m$ van a las posiciones que suman.

Scene5_Walk — Recorrido paso a paso sobre `nums = [1, -4, 2, -3, 5, -2]`

- Elevamos cada bloque al cuadrado y ordenamos: `sq` $= [1, 4, 4, 9, 16, 25]$.
- Como $n = 6$, la mitad es $m = 6//2 = 3$.
- Mitad izquierda (resta): `sq[:3]` $= [1, 4, 4] \implies -(1 + 4 + 4) = -9$.
- Mitad derecha (suma): `sq[3:]` $= [9, 16, 25] \implies +(9 + 16 + 25) = +50$.
- Total: $50 - 9 = 41$. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n \log n)$ por el ordenamiento. **Espacio:** $O(n)$ para la lista de cuadrados.

Scene7_Outro — Código en Python y lección

```python
class Solution:
    def maxAlternatingSum(self, nums: list[int]) -> int:
        sq = sorted(x * x for x in nums)
        m = len(sq) // 2
        return sum(sq[m:]) - sum(sq[:m])
```

Lección: *"El cuadrado borra los signos: ordena de menor a mayor, resta la primera mitad y suma la segunda."*

6 2026-10-14 | #3723 Maximize Sum of Squares of Digits

videos/math/3723-max-sum-squares-digits [Medium] · **Tema:** `math` (Greedy)

Scene1_Hook — Gancho y metáfora en Minecraft

Tenemos `num` casillas en el cofre y una suma total de `sum` puntos para repartir en dígitos del 0 al 9. Como el puntaje de cada casilla es su dígito al cuadrado, ¡un solo 9 vale más que un 5 y un 4 juntos!

Scene2_Problem — El problema y ejemplo pequeño (5 casillas)

Dados dos enteros positivos `num` (cantidad de dígitos) y `sum` (suma de sus dígitos), devolver como cadena el mayor entero positivo de `num` dígitos cuya suma sea `sum` y que maximice la suma de los cuadrados de sus dígitos (o `""` si no existe).

- **Ejemplo de 5 casillas:** `num = 5`, `sum = 21`.
- **Respuesta esperada:** `"99300"` (puntaje $9^2 + 9^2 + 3^2 + 0^2 + 0^2 = 171$).

Scene3_Brute — La idea directa (Programación dinámica por dígito) y por qué falla

Hacer una tabla DP de estados (posición, suma_restante). Con num $= 2 \times 10^5$ y sum $= 2 \times 10^6$, el número de estados es 4×10^{11} ($O(\text{num} \cdot \text{sum})$), lo cual agota memoria y tiempo.

Scene4_Idea — El truco (Convexidad de d^2 y división entera entre 9)

1. Si sum $>$ 9 $\times$ num (ni llenando todo con `'9'` alcanzamos la suma), es imposible y devolvemos `""`.
2. Si tenemos dos dígitos $a \geq b$ y pasamos 1 punto de b hacia a, la suma de cuadrados cambia en $(a+1)^2 + (b-1)^2 - (a^2 + b^2) = 2(a - b + 1) > 0$. Es decir, **concentrar todo lo posible en dígitos 9 siempre maximiza la suma de cuadrados**.
3. Para que además el número sea el mayor posible en valor posicional, ponemos todos los `'9'` a la izquierda, seguidos del residuo sum mod 9, y rellenamos con `'0'` a la derecha.

Scene5_Walk — Recorrido paso a paso sobre `num = 5, sum = 21`

- ¿Cabe en 5 casillas? $21 \leq 9 \times 5 = 45$ ✓.
- Dividimos entre 9: $q = 21//9 = 2$ nueves y residuo $r = 21 \bmod 9 = 3$.
- Casillas 0 y 1 reciben `'9'`; la casilla 2 recibe el residuo `'3'`; las $5 - 2 - 1 = 2$ casillas restantes reciben `'0'`.
- Resultado: `"99300"`. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(\text{num})$ para construir la cadena. **Espacio:** $O(\text{num})$ para la respuesta.

Scene7_Outro — Código en Python y lección

```
class Solution:
    def maxSumOfSquares(self, num: int, sum: int) -> str:
        if sum > 9 * num or sum < 1:
            return ""
        q, r = divmod(sum, 9)
        if q == num:
            return "9" * num
        return "9" * q + str(r) + "0" * (num - q - 1)
```

Lección: *"Por convexidad, exprime todos los nueves posibles a la izquierda y deja el residuo enseguida."*

7 2026-10-15 | #2913 Subarrays Distinct Element Sum of Squares I

videos/arrays-hashing/2913-subarrays-distinct-squares-i **[Easy]** · **Tema:** `arrays-hashing`

Scene1_Hook — Gancho y metáfora en Minecraft

Recorremos todos los tramos de una fila de menas. Para cada punto de partida abrimos un cofre (conjunto de menas vistas) y, a medida que alargamos el tramo hacia la derecha, sumamos el cuadrado de cuántas menas distintas hay en el cofre.

Scene2_Problem — El problema y ejemplo pequeño (5 bloques)

Dado un arreglo `nums` ($n \leq 100$), el conteo distinto de un subarreglo `nums[i..j]` es la cantidad de valores únicos que contiene. Devolver la suma de los cuadrados de ese conteo sobre todos los subarreglos.

- **Ejemplo de 5 bloques:** `nums = [2, 1, 2, 3, 2]`.
- **Respuesta esperada:** `65`.

Scene3_Brute — La idea directa ($O(n^3)$ reconstruyendo cada subarreglo)

Hacer tres ciclos: elegir i, elegir j, y calcular `len(set(nums[i:j+1])) ** 2` desde cero cada vez. Eso repite el trabajo de meter los mismos bloques una y otra vez ($O(n^3)$).

Scene4_Idea — El truco (Cofre incremental fijando el extremo izquierdo i)

Si ya conocemos el conjunto `seen` de distintos de `nums[i..j-1]`, para pasar a `nums[i..j]` solo insertamos `nums[j]` en `seen` en $O(1)$ y sumamos `len(seen) ** 2`. Eso baja el tiempo a $O(n^2)$.

Scene5_Walk — Recorrido paso a paso sobre `nums = [2, 1, 2, 3, 2]`

- $i = 0$: `seen` crece con $[2, 1, 2, 3, 2]$ → tamaños $1, 2, 2, 3, 3$ → cuadrados $1 + 4 + 4 + 9 + 9 = 27$.
- $i = 1$: desde `[1, 2, 3, 2]` → tamaños $1, 2, 3, 3$ → cuadrados $1 + 4 + 9 + 9 = 23$.
- $i = 2$: desde `[2, 3, 2]` → tamaños $1, 2, 2$ → cuadrados $1 + 4 + 4 = 9$.
- $i = 3$: desde `[3, 2]` → tamaños $1, 2$ → cuadrados $1 + 4 = 5$.
- $i = 4$: desde `[2]` → tamaño 1 → cuadrado 1.
- **Total:** $27 + 23 + 9 + 5 + 1 = 65$. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n^2)$. **Espacio:** $O(n)$ para el conjunto `seen`.

Scene7_Outro — Código en Python y lección

```
class Solution:
    def sumCounts(self, nums: list[int]) -> int:
        ans = 0
        n = len(nums)
        for i in range(n):
            seen = set()
            for j in range(i, n):
                seen.add(nums[j])
                ans += len(seen) ** 2
        return ans
```

Lección: *"No vacíes el cofre en cada paso: reutiliza el conjunto al extender el extremo derecho."*

8 2026-10-16 | #2916 Subarrays Distinct Element Sum of Squares II

videos/trees/2916-subarrays-distinct-squares-ii **[Hard]** · **Tema:** `trees` (Lazy Segment Tree)

Scene1_Hook — Gancho y metáfora en Minecraft

Es el hermano mayor del #2913: ahora la fila tiene hasta 100 000 bloques. En vez de extender un tramo a la vez, usamos un **árbol de bloques (Segment Tree)** que actualiza miles de subarreglos de un solo golpe.

Scene2_Problem — El problema y ejemplo pequeño (5 bloques)

Exactamente la misma pregunta que el #2913 (suma de los cuadrados del conteo de distintos de todos los subarreglos), pero con $n \le 10^5$ y devolviendo el resultado módulo $10^9 + 7$.

- **Ejemplo de 5 bloques:** `nums = [2, 1, 2, 3, 2]` → **Respuesta:** `65`.

Scene3_Brute — Por qué falla la idea de la Parte I ($O(n^2)$)

Con $n = 10^5$, hay $\frac{n(n+1)}{2} \approx 5 \times 10^9$ subarreglos. Recorrerlos uno por uno en $O(n^2)$ rompe el pico.

Scene4_Idea — El truco (Barrido por extremo derecho r + Identidad $(d+1)^2 = d^2 + 2d + 1$)

1. Avanzamos el extremo derecho r de 0 a $n-1$ y mantenemos un arreglo virtual $d[0..r]$, donde $d[l]$ es la cantidad de distintos en $\texttt{nums}[l..r]$.
2. Cuando llega $x = \texttt{nums}[r]$, cuya última aparición previa fue en el índice $p = \text{last}[x]$ (o -1 si no había salido):
   - Si el subarreglo empieza en $l \le p$, ya contenía a x, así que $d[l]$ **no cambia**.
   - Si empieza en $l \in [p+1..r]$, x es nuevo para él, así que $d[l]$ **aumenta en** $+1$.
3. Para sumar $+v$ a todo un rango $[a..b]$ en un Segment Tree y mantener tanto $S_1 = \sum d[l]$ como $S_2 = \sum d[l]^2$, usamos el binomio al cuadrado en $O(1)$ por nodo:

$$S_2' = \sum_{l=a}^{b} (d[l] + v)^2 = S_2 + 2v \cdot S_1 + (b - a + 1)v^2, \qquad S_1' = S_1 + (b - a + 1)v$$

Scene5_Walk — Recorrido paso a paso sobre `nums = [2, 1, 2, 3, 2]`

- $r = 0$ ($x = 2, p = -1$): suma $+1$ en $[0..0] \to d = [1] \to S_2 = 1$ (`ans = 1`).
- $r = 1$ ($x = 1, p = -1$): suma $+1$ en $[0..1] \to d = [2, 1] \to S_2 = 4 + 1 = 5$ (`ans = 6`).
- $r = 2$ ($x = 2, p = 0$): suma $+1$ en $[1..2] \to d = [2, 2, 1] \to S_2 = 4 + 4 + 1 = 9$ (`ans = 15`).
- $r = 3$ ($x = 3, p = -1$): suma $+1$ en $[0..3] \to d = [3, 3, 2, 1] \to S_2 = 9 + 9 + 4 + 1 = 23$ (`ans = 38`).
- $r = 4$ ($x = 2, p = 2$): suma $+1$ en $[3..4] \to d = [3, 3, 2, 2, 1] \to S_2 = 9 + 9 + 4 + 4 + 1 = 27$ (`ans = 65`).

Scene6_Complexity — Complejidad

Tiempo: $O(n \log n)$. **Espacio:** $O(n)$ para el Segment Tree y el diccionario `last`.

Scene7_Outro — Código en Python y lección

```
class Solution:
    def sumCounts(self, nums: list[int]) -> int:
        MOD = 10**9 + 7
        n = len(nums)
        s1, s2, lz = [0] * (4 * n), [0] * (4 * n), [0] * (4 * n)

        def apply(u, l, r, v):
```

```
            m = r - l + 1
            s2[u] = (s2[u] + 2 * v * s1[u] + m * v * v) % MOD
            s1[u] = (s1[u] + m * v) % MOD
            lz[u] += v

        def update(u, l, r, ql, qr):
            if ql <= l and r <= qr:
                apply(u, l, r, 1)
                return
            mid = (l + r) // 2
            if lz[u]:
                apply(u * 2, l, mid, lz[u])
                apply(u * 2 + 1, mid + 1, r, lz[u])
                lz[u] = 0
            if ql <= mid:
                update(u * 2, l, mid, ql, qr)
            if qr > mid:
                update(u * 2 + 1, mid + 1, r, ql, qr)
            s1[u] = (s1[u * 2] + s1[u * 2 + 1]) % MOD
            s2[u] = (s2[u * 2] + s2[u * 2 + 1]) % MOD

        last, ans = {}, 0
        for r, x in enumerate(nums):
            update(1, 0, n - 1, last.get(x, -1) + 1, r)
            last[x] = r
            ans = (ans + s2[1]) % MOD
        return ans
```

Lección: *"Un elemento nuevo en r solo incrementa los inicios posteriores a su última aparición; el binomio $(d+1)^2$ actualiza todos sus cuadrados a la vez."*

9 2026-10-17 | #3551 Minimum Swaps to Sort by Digit Sum

videos/arrays-hashing/3551-min-swaps-digit-sum **[Medium]** · **Tema:** `arrays-hashing` (Cycles)

Scene1_Hook — Gancho y metáfora en Minecraft

Cada bloque lleva su número y la suma de sus dígitos. Para ordenarlos con el menor número de intercambios (swaps), seguimos las flechas de "¿quién va en mi lugar?" formando ciclos cerrados.

Scene2_Problem — El problema y ejemplo pequeño (6 bloques)

Dado un arreglo `nums` de enteros positivos distintos, ordenarlo en orden creciente según la suma de sus dígitos (y en caso de empate, el número menor va primero). Devolver el mínimo número de swaps para lograrlo.

- **Ejemplo de 6 bloques:** `nums = [18, 43, 34, 16, 25, 11]`.
- **Claves (suma_dígitos, x):** $18 \to (9, 18)$, $43 \to (7, 43)$, $34 \to (7, 34)$, $16 \to (7, 16)$, $25 \to (7, 25)$, $11 \to (2, 11)$.
- **Orden destino:** $[11, 16, 25, 34, 43, 18]$ → **Respuesta:** 4 swaps.

Scene3_Brute — La idea directa (Selection Sort cuadrático) y por qué falla

Buscar en cada posición i el elemento mínimo en `nums`$[i..n-1]$ escaneando el resto del arreglo toma $O(n^2)$ comparaciones, que para $n = 10^5$ tarda demasiado.

Scene4_Idea — El truco (Descomposición en ciclos de una permutación)

1. Calculamos en $O(n \log n)$ el arreglo `order`, donde `order[i]` es el índice original del elemento que debe terminar en la posición i.
2. Vista como un grafo dirigido $i \to \text{order}[i]$, toda permutación se descompone en **ciclos disjuntos**.
3. Un ciclo de longitud L se resuelve con exactamente $L - 1$ swaps. Así, la respuesta es $\sum(L - 1) = n -$ (número de ciclos).

Scene5_Walk — Recorrido paso a paso sobre `nums = [18, 43, 34, 16, 25, 11]`

- Índices ordenados por clave: `order` $= [5, 3, 4, 2, 1, 0]$.
- **Ciclo 1:** $0 \leftrightarrow 5$ (el 18 y el 11 intercambian lugares): longitud $L = 2 \implies 2 - 1 = 1$ swap.
- **Ciclo 2:** $1 \to 3 \to 2 \to 4 \to 1$ $(43, 16, 34, 25)$: longitud $L = 4 \implies 4 - 1 = 3$ swaps.
- **Total:** $1 + 3 = 4$ swaps. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n \log n)$ para ordenar las claves y $O(n)$ para recorrer los ciclos. **Espacio:** $O(n)$.

Scene7_Outro — Código en Python y lección

```
class Solution:
    def minSwaps(self, nums: list[int]) -> int:
        def dsum(x: int) -> int:
            s = 0
            while x:
                s += x % 10
                x //= 10
            return s

        n = len(nums)
        order = sorted(range(n), key=lambda i: (dsum(nums[i]), nums[i]))
        vis = [False] * n
```

```
        swaps = 0
        for i in range(n):
            if vis[i] or order[i] == i:
                continue
            cycle = 0
            cur = i
            while not vis[cur]:
                vis[cur] = True
                cur = order[cur]
                cycle += 1
            swaps += cycle - 1
        return swaps
```

Lección: *"Cada ciclo de longitud L se arregla en $L-1$ swaps; cuenta los ciclos y tendrás el mínimo."*

10 2026-10-18 | #1569 Number of Ways to Reorder Array to Get Same BST

videos/trees/1569-ways-to-reorder-bst **[Hard]** · **Tema:** `trees` / `math` (Combinatorics)

Scene1_Hook — Gancho y metáfora en Minecraft

Insertamos bloques uno por uno para construir un Árbol Binario de Búsqueda (BST). El primer bloque siempre es la raíz, pero los bloques del lado izquierdo y los del lado derecho pueden entrelazarse en el cofre sin cambiar la forma del árbol.

Scene2_Problem — El problema y ejemplo pequeño (5 bloques)

Dada una permutación `nums` de $1..n$, encontrar cuántas reordenaciones distintas de `nums` producen exactamente el mismo BST al insertarse secuencialmente, módulo 10^9+7 (sin contar el arreglo original).

- **Ejemplo de 5 bloques:** `nums = [3, 4, 5, 1, 2]` (Raíz 3; izquierda $[1,2]$; derecha $[4,5]$).
- **Respuesta esperada:** `5` reordenaciones adicionales ($\binom{4}{2} \times 1 \times 1 - 1 = 6 - 1 = 5$).

Scene3_Brute — La idea directa (Generar permutaciones y armar el BST) y por qué falla

Probar las $n!$ permutaciones y comparar sus árboles toma $O(n! \cdot n \log n)$. Con $n = 1000$, es astronómico.

Scene4_Idea — El truco (Divide y vencerás + Entrelazado combinatorio $\binom{L+R}{L}$)

1. El primer elemento $r = \texttt{nums}[0]$ **debe** ir en la posición 0 para que sea la raíz.
2. Separamos el resto manteniendo su orden relativo: `left` $= [x \in \texttt{nums} : x < r]$ de tamaño L, y `right` $= [x \in \texttt{nums} : x > r]$ de tamaño R.
3. Como todo elemento menor que r cae a la izquierda de la raíz y todo elemento mayor cae a la derecha, las inserciones de `left` y de `right` son completamente independientes entre sí.
4. De las $L + R$ casillas después de la raíz, podemos elegir cualesquiera L casillas para colocar la secuencia `left` de $\binom{L+R}{L}$ maneras, y multiplicamos recursivamente por ways(`left`) $\times$ ways(`right`). Al final restamos 1 por el arreglo original.

Scene5_Walk — Recorrido paso a paso sobre `nums = [3, 4, 5, 1, 2]`

- Raíz fija: $r = 3$. Subsecuencia izquierda: `left` $= [1,2]$ ($L = 2$). Subsecuencia derecha: `right` $= [4,5]$ ($R = 2$).
- En `left` $= [1,2]$, el 1 debe ir antes del 2 $\implies$ ways$([1,2]) = 1$.
- En `right` $= [4,5]$, el 4 debe ir antes del 5 $\implies$ ways$([4,5]) = 1$.
- Formas de entrelazar 2 izquierdos y 2 derechos en 4 casillas: $\binom{2+2}{2} = \binom{4}{2} = 6$.
- Restamos el arreglo original: $6 - 1 = 5$. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n^2)$ (en cada nodo filtramos a sus descendientes). **Espacio:** $O(n)$ para las listas de la recursión.

Scene7_Outro — Código en Python y lección

```
from math import comb

class Solution:
    def numOfWays(self, nums: list[int]) -> int:
        MOD = 10**9 + 7

        def dfs(arr: list[int]) -> int:
```

```
            if len(arr) <= 2:
                return 1
            root = arr[0]
            left = [x for x in arr if x < root]
            right = [x for x in arr if x > root]
            ways = comb(len(left) + len(right), len(left)) % MOD
            return ways * dfs(left) % MOD * dfs(right) % MOD

        return (dfs(nums) - 1) % MOD
```

Lección: *"Fija la raíz primero y entrelaza los subárboles izquierdo y derecho con* $\binom{L+R}{L}$*."*

11 2026-10-19 | #2089 Find Target Indices After Sorting Array

videos/binary-search/2089-find-target-indices **[Easy]** · **Tema:** `binary-search` / `arrays-hashing`

Scene1_Hook — Gancho y metáfora en Minecraft

Queremos saber en qué casillas quedarán los bloques iguales al `target` si ordenamos la fila. Pero en lugar de ordenar toda la fila, ¡basta contar cuántos bloques son más pequeños que el `target`!

Scene2_Problem — El problema y ejemplo pequeño (6 bloques)

Dado un arreglo `nums` y un entero `target`, devolver la lista ordenada de índices donde `nums`$[i]$ == `target` después de ordenar `nums` en orden no decreciente.

- **Ejemplo de 6 bloques:** `nums = [5, 2, 1, 2, 3, 2]`, `target = 2`.
- **Respuesta esperada:** `[1, 2, 3]` (al ordenar queda $[1, \mathbf{2}, \mathbf{2}, \mathbf{2}, 3, 5]$).

Scene3_Brute — La idea directa (Ordenar todo el arreglo en $O(n \log n)$)

Llamar a `nums.sort()` y luego recorrer el arreglo buscando dónde quedó `target`. Funciona, pero gasta $O(n \log n)$ ordenando elementos mayores que `target` cuyo orden entre sí no nos importa en absoluto.

Scene4_Idea — El truco (Contar menores e iguales en una sola pasada $O(n)$)

1. En el arreglo ordenado, todos los elementos estrictamente menores que `target` ocuparán las primeras `less` casillas: los índices $0, 1, \ldots,$ `less` $- 1$.
2. Justo después irán todas las copias de `target`. Si hay `equal` copias de `target`, ocuparán exactamente los índices desde `less` hasta `less` + `equal` $- 1$.

Scene5_Walk — Recorrido paso a paso sobre `nums = [5, 2, 1, 2, 3, 2]`, `target = 2`

- Recorremos la fila una vez:
  - Menores que 2: solo el bloque 1 $\Longrightarrow$ `less` $= 1$.
  - Iguales a 2: tres bloques $\Longrightarrow$ `equal` $= 3$.
- Devolvemos `list(range(1, 1 + 3))` = `[1, 2, 3]`. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n)$. **Espacio auxiliar:** $O(1)$ (sin contar la lista de salida).

Scene7_Outro — Código en Python y lección

```python
class Solution:
    def targetIndices(self, nums: list[int], target: int) -> list[int]:
        less = 0
        equal = 0
        for x in nums:
            if x < target:
                less += 1
            elif x == target:
                equal += 1
        return list(range(less, less + equal))
```

Lección: *"Para saber tu lugar en la fila ordenada, solo cuenta cuántos van delante de ti."*

12 2026-10-20 | #3886 Sum of Sortable Integers

videos/arrays-hashing/3886-sum-of-sortable-integers **[Hard]** · **Tema:** `arrays-hashing` / `math`

Scene1_Hook — Gancho y metáfora en Minecraft

Partimos la fila de n bloques en tramos iguales de tamaño k y podemos girar cada tramo como un carrusel circular. Queremos sumar todos los tamaños k que logran dejar toda la fila ordenada.

Scene2_Problem — El problema y ejemplo pequeño (6 bloques)

Dado un arreglo `nums` de longitud n, un divisor k de n es *sortable* si al partir `nums` en subarreglos consecutivos de longitud k y rotar cíclicamente cada uno de forma independiente, todo el arreglo queda en orden no decreciente. Devolver la suma de todos los k válidos.

- **Ejemplo de 6 bloques:** `nums = [2, 3, 1, 5, 6, 4]` ($n = 6$, divisores $k \in \{1, 2, 3, 6\}$).
- **Respuesta esperada:** `3` (solo $k = 3$ funciona: $[2, 3, 1] \to [1, 2, 3]$ y $[5, 6, 4] \to [4, 5, 6]$).

Scene3_Brute — La idea directa (Rotar cada bloque k veces) y por qué falla

Para cada divisor k, probar las k rotaciones de cada bloque cuesta $O(k^2)$ por bloque, es decir $O(n \cdot k)$ por divisor. Si $k = n = 10^5$, un solo divisor toma 10^{10} operaciones.

Scene4_Idea — El truco (A lo sumo 1 caída circular por bloque + sumas de prefijos)

1. Un bloque $B = \texttt{nums}[s..e]$ puede ordenarse rotando cíclicamente **si y solo si** tiene a lo sumo un descenso al recorrerlo en círculo:
   - **Caso 0 caídas internas** ($\texttt{pref}[e] - \texttt{pref}[s] == 0$): ya está ordenado; su mínimo es $\texttt{nums}[s]$ y su máximo es $\texttt{nums}[e]$.
   - **Caso 1 caída interna** ($\texttt{pref}[e] - \texttt{pref}[s] == 1$) **y además** $\texttt{nums}[e] \le \texttt{nums}[s]$: si la única caída ocurrió en el índice p (donde $\texttt{nums}[p] > \texttt{nums}[p+1]$), al rotar el bloque su mínimo es $\texttt{nums}[p+1]$ y su máximo es $\texttt{nums}[p]$.
2. Además, entre bloques consecutivos debe cumplirse max(bloque anterior) $\le$ min(bloque actual).
3. Precalculando `pref` (conteo acumulado de caídas) y `last_drop` (índice de la última caída), revisamos cada bloque en $O(1)$, y todos los divisores de n toman en total $\sum_{k|n} \frac{n}{k} = O(n \log n)$.

Scene5_Walk — Recorrido paso a paso sobre `nums = [2, 3, 1, 5, 6, 4]`

- Caídas internas (`nums[i] > nums[i+1]`): en $i = 1$ ($3 > 1$) y en $i = 4$ ($6 > 4$).
- $k = 1$: bloques de 1 elemento; el bloque 1 vale 3 y el bloque 2 vale 1 ($3 > 1$) $\times$.
- $k = 2$: bloques $[2, 3], [1, 5], [6, 4]$; el primero tiene max $= 3 >$ min$([1, 5]) = 1$ $\times$.
- $k = 3$: bloque $[2, 3, 1]$ tiene 1 caída en $p = 1$ y $1 \le 2 \implies (\min, \max) = (1, 3)$; bloque $[5, 6, 4]$ tiene 1 caída en $p = 4$ y $4 \le 5 \implies (\min, \max) = (4, 6)$; además $3 \le 4$ ✓.
- $k = 6$: el bloque entero tiene 2 caídas internas $\times$. **Respuesta:** $k = 3$. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n \log n)$ (suma armónica sobre los divisores de n). **Espacio:** $O(n)$.

Scene7_Outro — Código en Python y lección

```
class Solution:
    def sortableIntegers(self, nums: list[int]) -> int:
        n = len(nums)
        pref = [0] * n
```

```
        last_drop = [-1] * n
        for i in range(n - 1):
            drop = 1 if nums[i] > nums[i + 1] else 0
            pref[i + 1] = pref[i] + drop
            last_drop[i + 1] = i if drop else last_drop[i]

        def check(k: int) -> bool:
            prev_max = -1
            for s in range(0, n, k):
                e = s + k - 1
                drops = pref[e] - pref[s]
                if drops == 0:
                    b_min, b_max = nums[s], nums[e]
                elif drops == 1 and nums[e] <= nums[s]:
                    p = last_drop[e]
                    b_min, b_max = nums[p + 1], nums[p]
                else:
                    return False
                if prev_max > b_min:
                    return False
                prev_max = b_max
            return True

        return sum(k for k in range(1, n + 1) if n % k == 0 and check(k))
```

Lección: *"Un tramo giratorio se ordena si y solo si tiene a lo sumo una caída circular."*

13 2026-10-21 | #3863 Minimum Operations to Sort a String

`videos/strings/3863-min-ops-sort-string` **[Medium]** · **Tema:** `strings` (Greedy — *Ya hecho en* ***`laptop-videos`***)

Scene1_Hook — Gancho y metáfora en Minecraft

En un paso podemos elegir cualquier tramo de la fila —**excepto la fila entera**— y ordenarlo. Parece un problema de búsqueda, pero la respuesta **nunca pasa de 3**.

Scene2_Problem — El problema y ejemplo del video (4 bloques)

Dada una cadena s, en una operación podemos elegir cualquier subcadena propia (longitud $< n$) y ordenarla alfabéticamente. Devolver el mínimo de operaciones para ordenar s, o -1 si es imposible.

- **Ejemplo del video (`laptop-videos`):** `s = "dbca"` (Menas: `d`→`diamond`, `b`→`lapis`, `c`→`coal`, `a`→`iron`).
- **Respuesta esperada:** `3`.

Scene3_Brute — La idea directa (BFS probando cada subcadena) y por qué falla

Probar en anchura ordenar cada una de las $\approx n^2/2$ subcadenas propias en cada nivel. Con $n = 10^5$, el árbol de búsqueda explota de inmediato.

Scene4_Idea — El truco (Clasificación exhaustiva en $\{0, -1, 1, 2, 3\}$ usando $\min(s)$ y $\max(s)$)

1. **0 operaciones:** Si s ya está ordenada.
2. -1 **(imposible):** Si $n = 2$ y está desordenada (toda subcadena propia mide 1, así que ordenarla no hace nada).
3. **1 operación:** Si la letra menor `mn = min(s)` ya está en $s[0]$ (ordenamos $s[1:]$) o la letra mayor `mx = max(s)` ya está en $s[-1]$ (ordenamos $s[:-1]$).
4. **2 operaciones:** Si `mn` aparece en algún lugar de $s[:-1]$ (ordenamos $s[:-1]$ para llevar `mn` al inicio, y luego ordenamos $s[1:]$) o si `mx` aparece en $s[1:]$.
5. **3 operaciones:** En el único caso restante (`mn` solo aparece al final y `mx` solo aparece al principio, como en `"dbca"`), ordenamos $s[:-1]$ para sacar a `mx` del inicio, luego $s[1:]$ para poner a `mx` al final y a `mn` adentro, y finalmente $s[:-1]$.

Scene5_Walk — Recorrido paso a paso sobre `s = "dbca"`

- ¿Ya ordenada? No. ¿Mide 2? No ($n = 4$). `mn` = `'a'`, `mx` = `'d'`.
- ¿Está `'a'` al inicio o `'d'` al final? No $\implies$ no basta 1 operación.
- ¿Aparece `'a'` en `"dbc"` o `'d'` en `"bca"`? No $\implies$ no bastan 2 operaciones.
- **Las 3 operaciones:** `"dbc" + "a"` → `"bcda"` → `"b" + "cda"` → `"bacd"` → `"bac" + "d"` → `"abcd"`. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n)$. **Espacio:** $O(1)$ (verificando orden con un barrido lineal).

Scene7_Outro — Código en Python y lección

```
class Solution:
    def minOperations(self, s: str) -> int:
        if all(s[i] <= s[i + 1] for i in range(len(s) - 1)):
            return 0
        if len(s) <= 2:
            return -1
        mn, mx = min(s), max(s)
```

```
        if s[0] == mn or s[-1] == mx:
            return 1
        if mn in s[:-1] or mx in s[1:]:
            return 2
        return 3
```

Lección: *"A veces el mejor algoritmo es una demostración: solo el mínimo y el máximo deciden entre 0, 1, 2 y 3."*

14 2026-10-22 | #3942 Minimum Operations to Sort a Permutation

videos/arrays-hashing/3942-min-ops-sort-permutation [Medium] · **Tema:** `arrays-hashing` (Cyclic / Reversal)

Scene1_Hook — Gancho y metáfora en Minecraft

Tenemos una permutación de bloques del 0 al $n-1$ y solo dos palancas: **rotar un paso a la izquierda** o **voltear toda la fila al revés**. ¿Cuántos movimientos faltan para dejarla ordenada?

Scene2_Problem — El problema y ejemplo pequeño (5 bloques)

Dada una permutación `nums` de $0..n-1$, en cada operación podemos: (1) invertir todo el arreglo (`Reverse`), o (2) pasar el primer elemento al final (`Rotate Left by One`). Devolver el mínimo de operaciones para llegar a $[0, 1, \ldots, n-1]$, o -1 si es imposible.

- **Ejemplo de 5 bloques:** `nums = [2, 3, 4, 0, 1]` ($n = 5$, el 0 está en el índice $p = 3$).
- **Respuesta esperada:** `3` (en vez de rotar 3 veces a la izquierda, o además empatando con $p = 3$; y para `[1, 0, 4, 3, 2]` con $p = 1$, rotar 2 veces a $[4, 3, 2, 1, 0]$ e invertir en 1 paso da 3).

Scene3_Brute — La idea directa (BFS clonando arreglos) y por qué falla

Hacer un BFS guardando arreglos de tamaño n visitados. Aunque solo hay $2n$ estados alcanzables, copiar e invertir arreglos de longitud $n = 10^5$ en cada paso toma $O(n^2)$ tiempo y memoria.

Scene4_Idea — El truco (Geometría del grupo diedral: solo importan 2 orientaciones del bloque 0)

1. Rotar e invertir nunca alteran la vecindad circular: el arreglo solo tiene solución si a partir del índice p donde está el 0 sube circularmente (`nums`$[(p+i) \bmod n] == i$) o baja circularmente (`nums`$[(p-i) \bmod n] == i$). Si ninguna se cumple, devolvemos -1.
2. **Si sube circularmente:**
   - Camino A (solo rotar a la izq.): toma p rotaciones.
   - Camino B (invertir → rotar → invertir): al invertir, la orientación se voltea; rotamos $(n-p) \bmod n$ veces y volvemos a invertir $\implies$ costo $(n-p) \bmod n + 2$.
3. **Si baja circularmente (está invertido):**
   - Camino C (rotar primero, invertir al final): llevamos el 0 al índice $n-1$ con $(p+1) \bmod n$ rotaciones a la izquierda y aplicamos 1 inversión $\implies$ costo $((p+1) \bmod n) + 1$.
   - Camino D (invertir primero, rotar después): invertimos (1 paso, el 0 pasa a $n-1-p$) y hacemos $n-1-p$ rotaciones $\implies$ costo $n-p$.

Scene5_Walk — Recorrido paso a paso sobre `nums = [1, 0, 4, 3, 2]` ($n = 5$)

- El bloque 0 está en $p = 1$. Verificamos `nums`$[(1-i) \bmod 5] == i$: para $i = 0, 1, 2, 3, 4$ da $0, 1, 2, 3, 4$ ✓ (orientación invertida).
- Camino C (rotar y luego invertir): $((1+1) \bmod 5) + 1 = 3$ operaciones.
- Camino D (invertir y luego rotar): $5 - 1 = 4$ operaciones.
- Mínimo: $\min(3, 4) = 3$. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n)$. **Espacio:** $O(1)$.

Scene7_Outro — Código en Python y lección

```python
class Solution:
    def minOperations(self, nums: list[int]) -> int:
        n = len(nums)
        if n <= 1:
            return 0
        p = nums.index(0)
        ans = float("inf")
        if all(nums[(p + i) % n] == i for i in range(n)):
            ans = min(ans, p, (n - p) % n + 2)
        if all(nums[(p - i) % n] == i for i in range(n)):
            ans = min(ans, (p + 1) % n + 1, n - p)
        return -1 if ans == float("inf") else ans
```

Lección: *"Dos inversiones seguidas se cancelan: toda ruta óptima usa a lo sumo una o dos inversiones."*

15 2026-10-23 | #3644 Maximum K to Sort a Permutation

videos/bit-manipulation/3644-max-k-sort-permutation **[Medium]** · **Tema:** `bit-manipulation` (*Ya hecho en* `main`)

Scene1_Hook — Gancho y metáfora en Minecraft

Siete bloques de mena del 0 al 6 revueltos, cada uno con tres lámparas de redstone debajo mostrando sus bits. Solo podemos intercambiar dos bloques cuando su `AND` bit a bit es exactamente k.

Scene2_Problem — El problema y ejemplo del video (7 bloques)

Dada una permutación `nums` de $0..n-1$, podemos intercambiar `nums`$[i]$ y `nums`$[j]$ solo si (`nums`$[i]$ AND `nums`$[j]$) == k. Devolver el máximo k que permite ordenar todo el arreglo (o 0 si ya está ordenado).

- **Ejemplo del video (`main`):** `nums = [0, 3, 2, 5, 4, 1, 6]`.
- **Bloques fuera de lugar (`nums`$[i] \neq i$):** 3 (011_2), 5 (101_2) y 1 (001_2).
- **Respuesta esperada:** $k = 3$ AND 5 AND $1 = 1$.

Scene3_Brute — La idea directa (Probar cada k y cada par de bloques) y por qué falla

Probar cada valor de k desde $n-1$ hacia abajo y para cada uno revisar los $\binom{n}{2}$ pares para armar componentes conexas cuesta $O(n^3)$. En el ejemplo de 7 bloques ya son $6 \times 21 = 126$ chequeos y el pico se rompe.

Scene4_Idea — El truco (El AND acumulado de los bloques fuera de lugar)

1. Todo bloque fuera de casa (`nums`$[i] \neq i$) **tiene que moverse**, así que debe participar en al menos un swap.
2. Para que un bloque x participe en un swap x AND $y = k$, todos los bits encendidos de k deben estar encendidos en x (es decir, k es subconjunto de los bits de x).
3. Por tanto, k debe caber dentro de **todos** los bloques descolocados. El mayor k que cumple eso es el `AND` bit a bit de todos los x con `nums`$[i] \neq i$ (y siempre es alcanzable usando al propio bloque k como pivote).

Scene5_Walk — Recorrido paso a paso sobre `nums = [0, 3, 2, 5, 4, 1, 6]`

- El panel de lámparas arranca totalmente encendido (`ans` $= -1$, todos los bits en 1).
- $i = 0$ (0): en casa, se salta.
- $i = 1$ ($3 = 011_2$): fuera de lugar $\Longrightarrow$ `ans` $= 011_2 = 3$ (se apaga la lámpara de 4).
- $i = 2$ (2): en casa, se salta.
- $i = 3$ ($5 = 101_2$): fuera de lugar $\Longrightarrow$ `ans` $= 011_2$ AND $101_2 = 001_2 = 1$ (se apaga la lámpara de 2).
- $i = 4$ (4): en casa. $i = 5$ ($1 = 001_2$): fuera de lugar $\Longrightarrow$ `ans` $= 001_2$ AND $001_2 = 1$.
- $i = 6$ (6): en casa. **Respuesta:** $k = 1$. *Advancement Made!*

Scene6_Complexity — Complejidad

Tiempo: $O(n)$ (una pasada y un `AND` por bloque). **Espacio:** $O(1)$.

Scene7_Outro — Código en Python y lección

```
class Solution:
    def sortPermutation(self, nums: list[int]) -> int:
        ans = -1
        for i, x in enumerate(nums):
            if x != i:
                ans &= x
```

```
        return max(ans, 0)
```

Lección: *"Solo los bloques fuera de lugar deciden k: su AND común es la respuesta."*